

Interface Specification ver_2.0

Interface Specification

이 문서는 Just Pick It 시스템에서 "누가 누구에게 무엇을 보내고, 무엇을 받는가"를 기준으로 인터페이스를 정리한다.

프로젝트 완성 기준은 이 문서를 따른다.

1. 작성 기준

항목	기준
UI 통신	Customer UI와 Admin UI는 Web Gateway의 HTTP/WebSocket API를 사용한다.
Web Gateway	화면 렌더링과 <code>/api/*</code> 프록시를 담당한다. DB에 직접 접근하지 않는다.
Fleet API	Fleet Manager 프로세스 내부에서 실행되는 HTTP/WebSocket API이다.
데이터 저장	DB 읽기/쓰기는 Fleet Manager의 FleetRepository가 담당한다. Web Gateway, UI, 로봇은 DB에 직접 접근하지 않는다.
주문 감지	TaskManager가 FleetRepository를 통해 <code>ORDER_WAIT</code> 주문을 주기적으로 확인한다.
진열 감지	TaskManager가 FleetRepository를 통해 <code>REQUESTED</code> 상태의 <code>display_item</code> 을 주기적으로 확인한다.
주문 task 생성/배정	TaskManager가 task 생성, robot unit 배정, sequence_no 부여를 담당한다.
진열 task 생성/배정	TaskManager가 <code>display_item</code> 을 읽어 진열 task 5개를 생성한다.

진열 batch	여러 <code>display_item</code> 이 같은 진열 흐름으로 표시될 수 있으며, 이때 <code>display_batch_id</code> 를 사용한다.
경로/교통 판단	TrafficManager가 PICKY 이동 경로와 zone 점유 가능 여부를 판단한다.
로봇 제어	PICKY와 COBOT은 Web Gateway와 직접 task를 주고받지 않고 Fleet Manager를 통해 제어된다.
로봇 상태 보고	PICKY/COBOT State Machine이 ROS2 Topic으로 상태를 발행하고, Fleet Manager가 수신해 DB에 반영한다.
LLM/STT	고객 음성 주문에서 STT로 텍스트를 만들고, LLM이 상품명과 수량을 구조화한다.
관리자 진열	관리자 진열은 LLM 자연어 명령이 아니라 Admin UI의 상품/수량 선택으로 생성한다.
Vision	Vision 요청은 이미지/프레임을 가진 로봇 쪽 State Machine 또는 Vision Node가 AI Vision Server에 직접 요청한다.
이미지/영상	Web Gateway와 Fleet API는 이미지/영상 프레임을 중계하지 않는다.
실시간 UI	Fleet API가 Customer/Admin UI 상태를 WebSocket으로 push하고, Web Gateway가 이를 프록시한다.
ROS2 통신	Fleet Manager와 로봇 State Machine은 ROS2 Topic/Action/Service로 통신한다.

용어 기준: 기존 문서의 "State Manager"는 이 문서에서 "State Machine"으로 통일한다.

2. 전체 흐름 요약

구분	흐름	핵심 역할
고객 상품 조회	Customer UI → Web Gateway → Fleet API	상품 목록 조회
고객 일반 주문	Customer UI → Web Gateway → Fleet API	장바구니 기반 주문 생성
고객 음성 주문	Customer UI → Web Gateway → STT → LLM → Fleet API	음성 입력을 주문 상품/수량으로 구조화 후 주문 생성
고객 주문 조회	Customer UI → Web Gateway → Fleet API	주문 상태, 픽업 슬롯 확인
관리자 관제	Admin UI → Web Gateway → Fleet API	로봇/주문/task/재고/픽업슬롯/예외 상태 조회 및 제어
관리자 진열 요청	Admin UI → Web Gateway → Fleet API	상품/수량 선택 기반 display_item 생성
자동 진열 요청	FleetRepository/TaskManager → DB	재고 부족 상태 진입 시 display_item 생성
주문 감지	TaskManager → FleetRepository → DB	ORDER_WAIT 주문 polling
주문 작업 생성	TaskManager → FleetRepository → DB	task 생성, robot unit 배정, 상태 전이
진열 작업 감지	TaskManager → FleetRepository → DB	REQUESTED display_item polling
진열 작업 생성	TaskManager → FleetRepository → DB	진열 task 생성, display_item 상태 변경, 진열 완료 처리
로봇 제어	Fleet Manager → PICKY/COBOT State Machine	이동, 상차, 검수, 하차, 진열, 복귀, 도킹, 충전, 긴급정지 명령

Vision 인식	PICKY/COBOT State Machine 또는 Vision Node → AI Vision Server	상품 인식, 검수, 진열 위치 탐색, 사람 감지
UI 실시간 갱신	Fleet API → Web Gateway → Customer/Admin UI	WebSocket 상태 push

3. 시스템 구성요소

1	Browser	
2	-> Web Gateway (:8000)	
3	-> Fleet API (:8100)	
4	-> FleetRepository	
5	-> just_pick_it_db	
6	-> PostgreSQL	
7		
8	FleetManagerNode	
9	├─ FleetRepository	DB 접근 단일 계층
10	├─ FleetApiServer	HTTP/WebSocket API
11	├─ TrafficManager	PICKY 경로 탐색/예약/충돌 회피
12	├─ RobotStateMonitor	로봇 텔레메트리 구독 -> DB + Traffic
13	├─ RobotCommandGateway	task -> ROS2 Action/Service 명령
14	└─ TaskManager	주문/진열 polling, task 생성/전이/dispatch

구성요소	책임
FleetManagerNode	컴포넌트 생성/연결, ROS2 Node, 주기 타이머 관리
FleetRepository	DB 읽기/쓰기 단일 진입점
FleetApiServer	Web Gateway가 호출하는 HTTP/WebSocket API 제공
TaskManager	주문/진열 감지, task 생성, robot unit 배정, 상태 전이
TrafficManager	zone graph 기반 경로 탐색, 예약, 해제
RobotCommandGateway	task를 로봇 ROS2 Action/Service 명령으로 변환해 송신
RobotStateMonitor	PICKY 상태/배터리/pose 토픽을 구독해 DB와 TrafficManager에 반영

Web Gateway	화면 제공, <code>/api/*</code> HTTP/WebSocket 프록시, 고객 STT/LLM 주문 route 담당
-------------	---

4. Customer UI ↔ Web Gateway / Fleet API

브라우저는 같은 origin의 Web Gateway를 호출하고, Web Gateway가 Fleet API로 프록시한다.

IF ID	호출 시 점	송신자	수신자	목적	요청	응답	프로토 콜
IF-C-01	상품 목록 화면 진입	Customer UI	Web Gateway → Fleet API	상품 목록 조회	GET <code>/api/products</code>	product 목록	HTTP
IF-C-02	상품 선택/수량 변경	Customer UI	Customer UI	장바구니 상태 변경	서버 API 없음	브라우저 local state	Local
IF-C-03	주문 확정	Customer UI	Web Gateway → Fleet API	주문 생성	POST <code>/api/orders</code>	order 상세	HTTP
IF-C-04	주문 현황 확인	Customer UI	Web Gateway → Fleet API	주문 상세 조회	GET <code>/api/orders/{order_id}</code>	order 상세	HTTP
IF-C-05	고객 화면 초기 진입/새 로그침	Customer UI	Web Gateway → Fleet API	고객 화면 전체 상태 조회	GET <code>/api/customer/status</code>	products, orders	HTTP

IF-C-06	주문/상품 상태 변경 시 자동	Fleet API → Web Gateway	Customer UI	고객 화면 실시간 상태 push	WS /api/customer/status	customer status	WebSocket
IF-C-07	고객 수령 완료	Customer UI	Web Gateway → Fleet API	주문 완료 처리	POST /api/orders/{order_id}/complete	completed order	HTTP
IF-C-08	고객 음성 주문	Customer UI	Web Gateway → STT/LLM → Fleet API	음성 주문 생성	POST /api/customer/llm/messages	LLM result + order	HTTP

주문 생성 요청 예시:

```

1 {
2   "items": [
3     {
4       "product_id": 1,
5       "quantity": 2,
6     },
7     {
8       "product_id": 3,
9       "quantity": 1,
10    }
11  ]
12 }
```

고객 음성 주문 요청 예시:

```

1 {
2   "message": "수박 두 개랑 생수 한 개 주문해줘"
3 }
```

고객 음성 주문 처리 결과 예시:

```
1 {
2   "result": "ok",
3   "message": "수박 2개, 생수 1개 주문 요청을 생성합니다.",
4   "action": "ORDER",
5   "items": [
6     {
7       "product_id": 1,
8       "product_name": "수박",
9       "quantity": 2
10    },
11    {
12      "product_id": 6,
13      "product_name": "생수",
14      "quantity": 1
15    }
16  ],
17  "order_id": 7,
18  "order_no": "ORD-0007",
19  "order": {
20    "order_id": 7,
21    "order_no": "ORD-0007",
22    "status": "ORDER_WAIT"
23  }
24 }
```

주문 생성 처리 원칙:

항목	처리
orders	생성 후 ORDER_WAIT
order_item	주문 상품 목록 생성
product	주문 수량만큼 stock_qty 차감
task	주문 생성 시점에는 생성하지 않음
Fleet Manager	이후 TaskManager가 ORDER_WAIT 주문을 감지해 task 생성
음성 주문	STT/LLM으로 상품명/수량을 구조화한 뒤 일반 주문 생성 API 흐름을 사용

5. Admin UI ↔ Web Gateway / Fleet API

관리자는 Web Gateway를 통해 Fleet API 상태를 조회하고 운영 상태를 수동 보정할 수 있다.

IF ID	호출 시 점	송신자	수신자	목적	요청	응답	프로토 콜
-------	-----------	-----	-----	----	----	----	----------

IF-A-01	관리자 대시보 드 진입	Admin UI	Web Gatewa y → Fleet API	전체 상 태 조회	GET /api/ admin /stat us	admin status snapsh ot	HTTP
IF-A-02	상태 변 경 시 자 동	Fleet API → Web Gatewa y	Admin UI	관리자 화면 실 시간 상 태 push	WS /api/ admin /ws/s tatu s	admin status snapsh ot	WebSo cket
IF-A-03	상품 생 성	Admin UI	Web Gatewa y → Fleet API	상품 추 가	POST /api/ admin /prod ucts	product	HTTP
IF-A-04	상품 수 정	Admin UI	Web Gatewa y → Fleet API	상품 정 보 수정	PATCH /api/ admin /prod ucts/ {prod uct_i d}	product	HTTP
IF-A-05	재고 수 정	Admin UI	Web Gatewa y → Fleet API	상품 재 고 수정	PATCH /api/ admin /prod ucts/ {prod uct_i d}/st ock	product	HTTP

IF-A-06	픽업 슬롯 생성	Admin UI	Web Gateway → Fleet API	pickup slot 추가	POST /api/admin/pickup-slots	pickup_slot	HTTP
IF-A-07	픽업 슬롯 상태 수정	Admin UI	Web Gateway → Fleet API	pickup slot 수동 보정	PATCH /api/fleet/pickup-slots/{slot_id}	status ok	HTTP
IF-A-08	task 상태 수정	Admin UI	Web Gateway → Fleet API	task 수동 보정	PATCH /api/fleet/tasks/{task_id}	status ok	HTTP
IF-A-09	로봇 상태 수정	Admin UI	Web Gateway → Fleet API	robot 수동 보정	PATCH /api/fleet/robots/{robot_id} 또는 robot_name	status ok	HTTP

IF-A-10	예외 해결	Admin UI	Web Gateway → Fleet API	exception 해결 처리	POST /api/admin/exceptions/{exception_id}/resolve	status ok	HTTP
IF-A-11	긴급 정지	Admin UI	Web Gateway → Fleet API → RobotCommandGateway	전체 로봇 정지	POST /api/admin/emergency-stop	status ok	HTTP + ROS2 Service
IF-A-12	재개	Admin UI	Web Gateway → Fleet API → RobotCommandGateway	긴급 정지 해제	POST /api/admin/resume	status ok	HTTP + ROS2 Service
IF-A-13	관리자 진열 요청	Admin UI	Web Gateway → Fleet API	display_item 생성	POST /api/admin/display-items	display_item	HTTP

IF-A-14	디버그 작업 완 료 처리	Admin UI	Web Gatewa y → Fleet API	로봇의 최근 RUNNI NG task 성 공 처리	POST <code>/api/ admin /debu g/rob ots/{ robot _iden tifie r}/ru nning - task/ succe ss</code>	status ok	HTTP
---------	---------------------	-------------	--------------------------------------	--	---	--------------	------

`PATCH /api/fleet/robots/{id}` 는 admin 수동 보정/검증용이다. 로봇 정기 상태 보고는 HTTP가 아니라 ROS2 topic으로 들어온다.

관리자 진열 요청 예시:

```

1 {
2   "product_id": 1,
3   "requested_quantity": 2,
4   "display_policy": "REQUESTED_QUANTITY"
5 }
```

디버그 작업 완료 처리 원칙:

항목	처리
대상	특정 robot의 최근 RUNNING task
제외	ASSIGNED/QUEUED task는 완료 처리 대상이 아님
용도	실제 로봇 없이 데모/디버그에서 현재 작업 성공 처리
UI	관리자 로봇 상태 영역의 현재 작업 클릭으로 호출 가능

운영	실제 운영 플로우에서는 Action result 가 정상 완료 신호가 되어야 함
----	---

6. Web Gateway ↔ Fleet API

Web Gateway는 `/api/*` 요청을 Fleet API로 전달한다.

```
1 | Browser -> http://localhost:8000/api/*
2 | Web Gateway -> http://localhost:8100/api/*
```

구분	Web Gateway 역할
HTML	<code>/customer</code> , <code>/admin</code> 등 화면 렌더링
HTTP API	<code>/api/{path:path}</code> 를 Fleet API로 프록시
WebSocket	<code>/api/{path:path}</code> WebSocket을 Fleet API WebSocket으로 프록시
고객 LLM	<code>/api/customer/llm/messages</code> 에서 STT/LLM 주문 파싱 후 ORDER이면 Fleet API에 주문 생성 요청
DB	직접 접근 금지

7. Fleet API HTTP Interface

7-1. 조회 API

엔드포인트	목적
GET <code>/api/health/db</code>	DB 연결 확인
GET <code>/api/products</code>	상품 목록 조회
GET <code>/api/orders</code>	주문 목록 조회
GET <code>/api/orders/{order_id}</code>	주문 상세 조회
GET <code>/api/customer/status</code>	Customer UI snapshot
GET <code>/api/admin/status</code>	Admin UI snapshot

GET <code>/api/fleet/snapshot</code>	Fleet 전체 snapshot
GET <code>/api/fleet/zones?</code> <code>zone_type={zone_type}</code>	zone 목록 조회
GET <code>/api/fleet/tasks</code>	task 목록 조회
GET <code>/api/fleet/orders?</code> <code>status={status}</code>	주문 목록 조회
GET <code>/api/fleet/orders/{order_id}</code> <code>}/tasks</code>	주문별 task 조회
GET <code>/api/fleet/pickup-</code> <code>slots?status={status}</code>	pickup slot 목록 조회

7-2. 명령 API

엔드포인트	목적
POST <code>/api/customer/llm/messages</code>	고객 음성/텍스트 주문 LLM 처리
POST <code>/api/orders</code>	주문 생성
POST <code>/api/orders/{order_id}/comp</code> <code>lete</code>	고객 수령 완료
POST <code>/api/admin/products</code>	상품 생성
PATCH <code>/api/admin/products/{produc</code> <code>t_id}</code>	상품 수정
PATCH <code>/api/admin/products/{produc</code> <code>t_id}/stock</code>	재고 수정
POST <code>/api/admin/pickup-</code> <code>slots</code>	픽업 슬롯 생성

POST /api/admin/display-items	진열 요청 생성
POST /api/admin/exceptions/{exception_id}/resolve	예외 해결
POST /api/admin/emergency-stop	긴급 정지
POST /api/admin/resume	재개
POST /api/admin/debug/robots/{robot_identifier}/running-task/success	디버그용 최근 RUNNING task 성공 처리
POST /api/fleet/tasks/bulk	task bulk 생성
PATCH /api/fleet/tasks/{task_id}	task 상태 수정
DELETE /api/fleet/tasks/{task_id}	task 삭제
PATCH /api/fleet/orders/{order_id}	주문 상태/픽업 슬롯 보정
PATCH /api/fleet/robots/{robot_id 또는 robot_name}	로봇 상태 수동 보정
PATCH /api/fleet/pickup-slots/{slot_id}	pickup slot 상태 수정

8. Fleet API WebSocket

IF ID	송신자	수신자	목적	경로
-------	-----	-----	----	----

IF-WS-01	Fleet API → Web Gateway	Admin UI	관리자 상태 push	WS /api/admi n/ws/stat us
IF-WS-02	Fleet API → Web Gateway	Customer UI	고객 상태 push	WS /api/cust omer/ws/s tatus

처리 원칙:

- WebSocket 연결 시 즉시 snapshot 1회를 전송한다.
- 이후 `api_push_interval_sec` 기본 1초마다 전체 snapshot을 push한다.
- Web Gateway는 WebSocket 메시지를 변형하지 않고 프록시한다.
- 현재는 delta event가 아니라 full snapshot 방식이다.
- 관리자 snapshot에는 주문, 진열 batch, task, 로봇 상태, 배터리, 예외 상태가 포함된다.

9. 고객 STT / LLM 주문 Interface

IF ID	호출 시 점	송신자	수신자	목적	요청	응답
IF-L-01	고객 음성 주문	Customer UI	Web Gateway	음성/텍스트 주문 메시지 전송	POST /api/customer/llm/messages	LLM result
IF-L-02	LLM 결과 ORDER	Web Gateway	Fleet API	주문 생성	POST /api/orders	order

요청 예시:

```
1 {
2   "message": "수박 두 개랑 생수 한 개 주문해줘"
3 }
```

오디오 data URL 요청 예시:

```

1 {
2   "message": "data:audio/webm;base64,AAAA..."
3 }

```

LLM 처리 결과 예시:

```

1 {
2   "result": "ok",
3   "message": "수박 2개, 생수 1개 주문 요청을 생성합니다.",
4   "action": "ORDER",
5   "items": [
6     {
7       "product_id": 1,
8       "product_name": "수박",
9       "quantity": 2
10    },
11    {
12      "product_id": 6,
13      "product_name": "생수",
14      "quantity": 1
15    }
16  ],
17  "provider": "gpt-4o-mini"
18 }

```

주문 생성 포함 응답 예시:

```

1 {
2   "result": "ok",
3   "message": "주문이 생성되었습니다. order_id=7, order_no=ORD-0007",
4   "action": "ORDER",
5   "items": [
6     {
7       "product_id": 1,
8       "product_name": "수박",
9       "quantity": 2
10    }
11  ],
12  "order_id": 7,
13  "order_no": "ORD-0007",
14  "order": {
15    "order_id": 7,
16    "order_no": "ORD-0007",
17    "status": "ORDER_WAIT"
18  }
19 }

```

처리 원칙:

- LLM 담당 모듈은 task를 직접 만들지 않는다.
- 오디오 입력이면 STT를 먼저 수행한 뒤 텍스트를 LLM에 전달한다.
- 텍스트 입력이면 바로 LLM에 전달한다.
- LLM은 판매 상품 목록 기준으로 상품명과 수량을 구조화한다.
- LLM 결과가 ORDER 이면 Web Gateway가 Fleet API의 /api/orders 로 주문 생성을 요청한다.
- 목록에 없는 상품은 주문 항목에서 제외한다.

- 수량이 없으면 기본 1개로 해석한다.
- 관리자 진열 명령에는 LLM을 사용하지 않는다.

10. Vision Interface

Vision 요청은 이미지/프레임이 발생하는 로봇 쪽에서 직접 수행한다. Web Gateway와 Fleet API는 이미지/영상 데이터를 중계하지 않는다.

IF ID	호출 시 점	송신자	수신자	목적	요청 데 이터	응답 데 이터	프로토 콜
IF-V-01	SORTING_AND_LOADING 수행 중	COBOT State Machine 또는 Vision Node	AI Vision Server	상품 인식 및 좌표 추정	task_id, product_name, camera_frame	object_pose, confidence	TBD
IF-V-02	INSPECTION 수행 중	COBOT State Machine 또는 Vision Node	AI Vision Server	주문 상품 검수	task_id, order_id, camera_frame	inspection_result, wrong_items	TBD
IF-V-03	DISPLAY_SCAN 수행 중	COBOT State Machine 또는 Vision Node	AI Vision Server	진열대 빈자리 탐색	task_id, display_item_id, camera_frame	empty_position, confidence	TBD
IF-V-04	사람 감지 필요 시	Vision Node 또는 State Machine	AI Vision Server	사람 감지	robot, camera_frame, zone	person_detected, confidence	TBD

항목	원칙
상품 인식 성공	COBOT State Machine이 작업을 진행하고 Fleet Manager에 result 반환
상품 인식 실패	COBOT State Machine이 action result success=false 반환
처리 수량	비전이 한 번에 계산한 감지 수량이 아니라, COBOT이 실제 집기/놓기/검수 완료한 수량을 processed_quantity로 누적
사람 감지	emergency stop 또는 safety stop 정책으로 연결
이미지 중계	Web Gateway와 Fleet API는 이미지/영상 프레임을 중계하지 않음
세부 프로토콜	Vision 담당 모듈에서 HTTP/gRPC/ROS2 Service 중 확정

11. Fleet Manager ↔ ROS2 / 로봇 State Machine

Fleet Manager는 RobotCommandGateway를 통해 PICKY/COBOT State Machine에 실제 작업을 지시한다. State Machine은 로봇 컨트롤러와 통신하고, 결과를 Fleet Manager에 반환한다.

용도	방식
PICKY 이동/복귀 명령	ROS2 Action
PICKY 도킹 명령	ROS2 Action
COBOT 선별/상차/검수/하차/진열 명령	ROS2 Action
PICKY 상태/배터리/pose 보고	ROS2 Topic
COBOT 상태 보고	ROS2 Topic
작업 진행 feedback	ROS2 Action feedback
작업 완료/실패 result	ROS2 Action result

긴급 정지/재개	ROS2 Service
----------	--------------

12. PICKY ROS2 Interface

robot_name	ROS namespace	미니맵 표시
PICKY1	/picky1	표시
PICKY2	/picky2	표시

robot_id DB PK는 seed/reset에 따라 달라질 수 있으므로 인터페이스 계약에서는 안정 식별자인 robot_name과 ros_namespace를 우선 사용한다.

담당 task_type:

task_type	설명
MOVE_TO_PRODUCT	주문 상품 보관 구역으로 이동
MOVE_TO_PICKUP	픽업존으로 이동
MOVE_TO_STOCK	창고 구역으로 이동
MOVE_TO_DISPLAY	진열 구역으로 이동
RETURN_HOME	standby zone 복귀
DOCK_IN	충전 도크 정밀 진입
CHARGE	충전 상태 진입/유지

PICKY 인터페이스 목록:

IF ID	호출 시 점	송신자	수신자	목적	ROS2 이 름	ROS2 타 입
IF-P-01	이동 task 실행 시	Fleet Manager	PICKY State Machine	waypoint 이동 명 령	/pick y_ns}/ move_c omman d	just_p ick_it _inter faces/ action /MoveC

						ommand
IF-P-02	도킹 task 실행 시	Fleet Manager	PICKY State Machine	충전 도크 정밀 진입	/picky_ns/dock_command	just_pick_interfaces/action/DockCommand
IF-P-03	상태 변경 시	PICKY State Machine	Fleet Manager	상태 보고	/picky_ns/picky_state	std_msgs/String
IF-P-04	배터리 변경 시	pinky_brngup	Fleet Manager	배터리 보고	/picky_ns/battery_percent	std_msgs/Float32
IF-P-05	pose 변경 시	AMCL	Fleet Manager	위치 보고	/picky_ns/amcl_pose	geometry_msgs/PoseWithCovarianceStamped
IF-P-06	긴급 정지/해제 시	Fleet Manager	PICKY State Machine	긴급 정지/해제	/picky_ns/emergency_control	just_pick_interfaces/srv/EmergencyControl

MoveCommand.action:

```
1 # Goal
2 string task_type
3 geometry_msgs/PoseStamped[] waypoints
4 ---
5 # Result
6 bool success
7 string message
8 ---
9 # Feedback
10 uint8 current_waypoint_index
11 uint8 total_waypoints
```

MoveCommand goal 예시:

```
1 {
2   "task_type": "MOVE_TO_PRODUCT",
3   "waypoints": [
4     {
5       "header": {
6         "frame_id": "map"
7       },
8       "pose": {
9         "position": {
10          "x": 1.2,
11          "y": 0.6,
12          "z": 0.0
13        },
14        "orientation": {
15          "z": 0.0,
16          "w": 1.0
17        }
18      }
19    }
20  ]
21 }
```

DockCommand.action:

```
1 # Goal
2 int32 task_id
3 string dock_name
4 string start_zone_name
5 ---
6 # Result
7 bool success
8 string message
9 ---
10 # Feedback
11 string phase
12 float32 progress
13 string message
```

PICKY 처리 원칙:

- RobotCommandGateway가 task_id와 action goal을 내부적으로 매핑한다.
- MoveCommand.action goal 자체에는 task_id가 없다.
- action feedback의 current_waypoint_index는 TrafficManager.update_path_progress()에 전달한다.

- action result success=true이면 task SUCCESS로 처리한다.
- action result success=false이면 task FAILED 및 exception_log를 기록한다.
- DOCK_IN은 move_command가 아니라 dock_command를 사용한다.
- PICKY 도착 상태는 재시작 복구 판단에 사용되므로 명확해야 한다.

PICKY state 계약:

task_type	실행 중 picky_state	도착/완료 시 picky_state
MOVE_TO_PRODUCT	MOVING_TO_PRODUCT	WAITING_FOR_COBOT
MOVE_TO_PICKUP	MOVING_TO_PICKUP	WAITING_FOR_COBOT
MOVE_TO_STOCK	MOVING_TO_STOCK	WAITING_FOR_COBOT
MOVE_TO_DISPLAY	MOVING_TO_DISPLAY	WAITING_FOR_COBOT
RETURN_HOME	RETURNING	STANDBY
DOCK_IN	DOCKING	CHARGING
CHARGE	CHARGING	CHARGING

13. COBOT ROS2 Interface

robot_name	ROS namespace	미니맵 표시
COBOT1	/cobot1	표시 안 함
COBOT2	/cobot2	표시 안 함

담당 task_type:

task_type	설명
SORTING_AND_LOAD	주문/진열 상품 선별 및 PICKY 상차
INSPECTION	주문 상품 검수

UNLOAD	픽업 슬롯 하차
DISPLAY_SCAN	진열대 빈자리 탐색
DISPLAY_PLACE	진열 상품 배치

COBOT 인터페이스 목록:

IF ID	호출 시 점	송신자	수신자	목적	ROS2 이 름	ROS2 타 입
IF-CB-01	COBOT task 실행 시	Fleet Manager	COBOT State Machine	COBOT 작업 명 령	<code>/cobot_ns/execute_task</code>	<code>just_pick_interfaces/action/ExecuteTask</code>
IF-CB-02	긴급 정 지/해제 시	Fleet Manager	COBOT State Machine	긴급 정 지/해제	<code>/cobot_ns/emergency_control</code>	<code>just_pick_interfaces/srv/EmergencyControl</code>
IF-CB-03	COBOT 상태 변 경 시	COBOT State Machine	Fleet Manager	COBOT 상태 보 고	<code>/cobot_ns/cobot_state</code>	<code>std_msgs/String</code>

COBOT은 ExecuteTask.action을 통해 작업 명령을 받고, `/cobotX/cobot_state` 로 세부 상태를 보고한다.

ExecuteTask.action 최종 계약:

```

1 # Goal
2 int32 task_id
3 string task_type

```

```

4 string product_name
5 int32 quantity
6 int32 order_id
7 int32 display_item_id
8 string target_zone_name
9 ---
10 # Result
11 bool success
12 string status
13 string message
14 int32 processed_quantity
15 int32 stock_delta
16 ---
17 # Feedback
18 string state
19 string message
20 float32 progress
21 int32 processed_quantity

```

Goal 필드 기준:

필드	의미
task_id	Fleet DB task id
task_type	SORTING_AND_LOAD, INSPECTION, UNLOAD, DISPLAY_SCAN, DISPLAY_PLACE
product_name	상품명. 결과 메시지와 작업 컨텍스트 에 사용
quantity	이번 작업에서 처리할 요청 수량
order_id	주문 task이면 order id, 없으면 0
display_item_id	진열 task이면 display_item id, 없으 면 0
target_zone_name	하차/진열 위치 이름. 없으면 빈 문자 열

robot_name은 Goal에 넣지 않는다. Fleet Manager가 이미 `/cobot_ns/execute_task` namespace로 대상 COBOT을 지정한다.

source_zone_name은 Goal에 넣지 않는다. 시연/실작업 기준으로 COBOT은 사람이 배치한 물체 또는 카메라 비전 결과를 기준으로 집기 위치를 판단한다.

Result 필드 기준:

필드	의미
success	작업 성공 여부. Fleet Manager의 기본 성공/실패 판단에 사용
status	SUCCESS, FAILED, CANCELLED
message	관리자 UI에 표시되는 최종 결과 문장. 실패 시 exception detail로도 사용
processed_quantity	COBOT이 실제 집기/놓기/검수 완료한 수량
stock_delta	DISPLAY_PLACE 성공 시 재고에 반영할 수량. 그 외 작업은 0

success=false이고 status=CANCELLED이면 task는 CANCELLED로 처리한다.

success=false이고 status=FAILED이면 task는 FAILED로 처리하고 exception log를 남긴다.

Feedback 필드 기준:

필드	의미
state	COBOT 현재 세부 상태. STOWING_ARM은 필수
message	진행 상황 메시지
progress	0.0 ~ 1.0 진행률
processed_quantity	현재까지 실제 처리 완료한 수량

STOWING_ARM feedback은 다음 PICKY 이동 선계획 트리거로 사용한다.

STOWING_ARM 진입 시 progress는 0.85 ~ 0.9 권장이다. 이는 물리 진행률이 아니라 관리자 UI/선예약용 단계값이다.

작업 result가 성공 처리되면 최종 진행률은 100%로 본다.

ExecuteTask goal 예시 - SORTING_AND_LOAD 주문:

```

1 {
2   "task_id": 102,
3   "task_type": "SORTING_AND_LOAD",
4   "product_name": "수박",
5   "quantity": 2,
6   "order_id": 7,
7   "display_item_id": 0,

```

```
8   "target_zone_name": ""
9 }
```

ExecuteTask goal 예시 - INSPECTION:

```
1 {
2   "task_id": 108,
3   "task_type": "INSPECTION",
4   "product_name": "",
5   "quantity": 2,
6   "order_id": 7,
7   "display_item_id": 0,
8   "target_zone_name": ""
9 }
```

ExecuteTask goal 예시 - UNLOAD:

```
1 {
2   "task_id": 109,
3   "task_type": "UNLOAD",
4   "product_name": "",
5   "quantity": 2,
6   "order_id": 7,
7   "display_item_id": 0,
8   "target_zone_name": "PICKUP_SLOT_1"
9 }
```

ExecuteTask goal 예시 - SORTING_AND_LOAD 진열:

```
1 {
2   "task_id": 203,
3   "task_type": "SORTING_AND_LOAD",
4   "product_name": "수박",
5   "quantity": 2,
6   "order_id": 0,
7   "display_item_id": 1,
8   "target_zone_name": ""
9 }
```

ExecuteTask goal 예시 - DISPLAY_SCAN:

```
1 {
2   "task_id": 205,
3   "task_type": "DISPLAY_SCAN",
4   "product_name": "수박",
5   "quantity": 2,
6   "order_id": 0,
7   "display_item_id": 1,
8   "target_zone_name": "PRODUCT_SLOT_1"
9 }
```

ExecuteTask goal 예시 - DISPLAY_PLACE:

```
1 {
2   "task_id": 206,
3   "task_type": "DISPLAY_PLACE",
4   "product_name": "수박",
5   "quantity": 2,
6   "order_id": 0,
7   "display_item_id": 1,
8   "target_zone_name": "PRODUCT_SLOT_1"
9 }
```

ExecuteTask feedback 예시:

```
1 {  
2   "state": "STOWING_ARM",  
3   "message": "팔 복귀 중",  
4   "progress": 0.85,  
5   "processed_quantity": 2  
6 }
```

ExecuteTask result 예시 - 성공:

```
1 {  
2   "success": true,  
3   "status": "SUCCESS",  
4   "message": "수박 2개 진열 완료",  
5   "processed_quantity": 2,  
6   "stock_delta": 2  
7 }
```

ExecuteTask result 예시 - 실패:

```
1 {  
2   "success": false,  
3   "status": "FAILED",  
4   "message": "수박 파지 실패",  
5   "processed_quantity": 1,  
6   "stock_delta": 0  
7 }
```

ExecuteTask result 예시 - 취소:

```
1 {  
2   "success": false,  
3   "status": "CANCELLED",  
4   "message": "작업 취소됨",  
5   "processed_quantity": 1,  
6   "stock_delta": 0  
7 }
```

Result.message 포맷:

작업	성공 메시지
SORTING_AND_LOAD 주문	{상품명} {처리수량}개 선별/상차 완료
SORTING_AND_LOAD 진열	{상품명} {처리수량}개 진열용 선 별/상차 완료
INSPECTION	주문 #{order_id} 검수 완료
UNLOAD	주문 #{order_id} {target_zone_name} 하차 완 료

DISPLAY_SCAN	{상품명} 진열대 스캔 완료
DISPLAY_PLACE	{상품명} {처리수량}개 진열 완료
상품 관련 실패	{상품명} {실패사유}
주문 관련 실패	주문 #{order_id} {실패사유}
취소	작업 취소됨

Result.message 생성 예시:

```

1 def build_success_message(goal, processed_quantity: int) -> str:
2     product_name = goal.product_name or "상품"
3     quantity = processed_quantity or goal.quantity
4     target = goal.target_zone_name or "목적지"
5
6     if goal.task_type == "SORTING_AND_LOAD":
7         if goal.display_item_id:
8             return f"{product_name} {quantity}개 진열용 선별/상차 완료"
9             return f"{product_name} {quantity}개 선별/상차 완료"
10
11     if goal.task_type == "INSPECTION":
12         return f"주문 #{goal.order_id} 검수 완료"
13
14     if goal.task_type == "UNLOAD":
15         return f"주문 #{goal.order_id} {target} 하차 완료"
16
17     if goal.task_type == "DISPLAY_SCAN":
18         return f"{product_name} 진열대 스캔 완료"
19
20     if goal.task_type == "DISPLAY_PLACE":
21         return f"{product_name} {quantity}개 진열 완료"
22
23     return f"{goal.task_type} 완료"
24
25
26 def build_failure_message(goal, reason: str) -> str:
27     reason = reason or "작업 실패"
28
29     if goal.task_type in ("INSPECTION", "UNLOAD") and goal.order_id:
30         return f"주문 #{goal.order_id} {reason}"
31
32     return f"{goal.product_name or '상품'} {reason}"

```

COBOT 처리 원칙:

- COBOT State Machine은 ExecuteTask.action goal을 받아 작업을 수행한다.
- Vision 요청이 필요하면 COBOT State Machine 또는 Vision Node가 AI Vision Server에 직접 요청한다.
- 본동작이 끝난 직후 바로 SUCCESS를 반환하지 않는다.
- 팔 복귀가 필요한 작업은 STOWING_ARM 상태를 feedback으로 Fleet Manager에 알려야 한다.
- Fleet Manager는 STOWING_ARM feedback을 받으면 다음 PICKY 이동을 선계획할 수 있다.

- 팔 복귀까지 완료된 뒤 action result success=true, status=SUCCESS를 반환한다.
- action result success=false, status=FAILED이면 Fleet Manager가 task FAILED 및 exception_log를 기록한다.
- action result success=false, status=CANCELLED이면 Fleet Manager가 task CANCELLED로 처리한다.
- DISPLAY_PLACE result의 processed_quantity, stock_delta는 진열 완료 처리에 사용한다.
- ExecuteTask.action server가 없거나 goal이 reject되면 TaskManager는 해당 task를 재시도 또는 실패 대상으로 처리한다.
- 실패를 숨기고 SUCCESS를 반환하면 안 된다.

COBOT state 계약:

task_type	실행 중 cobot_state	팔 복귀
SORTING_AND_LOAD	SORTING 또는 LOADING	STOWING_ARM
INSPECTION	INSPECTING	STOWING_ARM
UNLOAD	UNLOADING	STOWING_ARM
DISPLAY_SCAN	SCANNING	STOWING_ARM
DISPLAY_PLACE	PLACING	STOWING_ARM

STOWING_ARM 선계획 계약:

STOWING_ARM trigger	Fleet Manager 선계획
SORTING_AND_LOAD 주문	남은 상품 있으면 다음 MOVE_TO_PRODUCT, 없으면 MOVE_TO_PICKUP
SORTING_AND_LOAD 진열	다음 MOVE_TO_DISPLAY
INSPECTION	선계획 없음
UNLOAD	작업 종료 후 복귀/도킹 판단
DISPLAY_SCAN	다음 DISPLAY_PLACE는 이미 생성 되어 있으므로 순서 gate만 확인
DISPLAY_PLACE	작업 종료 후 복귀/도킹 판단

COBOT 인터페이스 결론 요약:

구분	필드
반드시 필요	Goal.task_id, Goal.task_type, Result.success, Result.status, Result.message, Feedback.state
관리자 UI 메시지 품질 때문에 필요	Goal.product_name, Goal.quantity, Goal.order_id, Goal.display_item_id, Goal.target_zone_name
진행률 표시 때문에 필요	Feedback.progress
처리 수량/진열 재고 반영 때문에 필요	Result.processed_quantity, Result.stock_delta
현재는 보조 값	Feedback.message, Feedback.processed_quantity

14. Emergency Stop Service

항목	값
Service name	<code>/robot_namespace/emergency_control</code>
Type	<code>just_pick_it_interfaces/srv/EmergencyControl</code>
<code>emergency_stop=true</code>	긴급 정지
<code>emergency_stop=false</code>	긴급 정지 해제 요청

EmergencyControl.srv:

```
1 bool emergency_stop
2 string reason
3 int32 task_id
4 string request_id
5 ---
6 bool accepted
7 string status
8 string message
```

처리 원칙:

- Admin UI가 emergency-stop/resume 요청을 Web Gateway에 보낸다.
- Web Gateway가 Fleet API로 요청을 전달한다.
- FleetManagerNode가 DB 상태 전이를 수행한다.
- RobotCommandGateway가 각 로봇 State Machine의 emergency_control service를 호출한다.
- emergency 중 새 dispatch는 막는다.
- emergency 중 로봇 action은 성공으로 처리하면 안 된다.
- 재개 불가 시 accepted=false와 사유를 반환한다.
- 실패한 로봇은 exception_log 또는 로그로 추적한다.

15. Robot ID / Namespace

robot_name	robot_type	robot_unit	ROS namespace	미니맵 표시
PICKY1	PICKY	PICKY_UNIT_1	/picky1	표시
COBOT1	COBOT	PICKY_UNIT_1	/cobot1	표시 안 함
PICKY2	PICKY	PICKY_UNIT_2	/picky2	표시
COBOT2	COBOT	PICKY_UNIT_2	/cobot2	표시 안 함

처리 원칙:

- DB PK인 robot_id는 seed/reset에 따라 달라질 수 있으므로 외부 계약에서는 고정값으로 의존하지 않는다.
- API path에서는 robot_id 또는 robot_name을 받을 수 있으나, 운영 문서에서는 robot_name을 안정 식별자로 사용한다.
- ROS namespace는 robot_name.lower() 기준이다. 예: PICKY1 → /picky1 .
- 미니맵에는 robot_type=PICKY만 이동 객체로 표시한다.

16. Zone / Slot 기준

구분	의미
----	----

ZONE	PICKY가 이동하거나 정차하는 위치
SLOT	COBOT이 물건을 집거나 내려놓는 실제 작업 위치

주요 매핑:

PICKY 정차 위치	COBOT 작업 위치
PRODUCT_ZONE_1	PRODUCT_SLOT_1
PRODUCT_ZONE_2	PRODUCT_SLOT_2
PRODUCT_ZONE_3	PRODUCT_SLOT_3
PRODUCT_ZONE_4	PRODUCT_SLOT_4
PRODUCT_ZONE_5	PRODUCT_SLOT_5
PRODUCT_ZONE_6	PRODUCT_SLOT_6
PICKUP_ZONE_1	PICKUP_SLOT_1
PICKUP_ZONE_2	PICKUP_SLOT_2
STOCK_ZONE	STOCK_SLOT

처리 원칙:

- PICKY 이동 task의 target은 ZONE을 사용한다.
- DB task에는 source/target zone을 저장해 UI 표시와 Fleet 흐름 추적에 사용한다.
- COBOT ExecuteTask.Goal에는 전체 source/target 좌표를 전달하지 않는다.
- COBOT은 실제 집기 위치를 카메라/비전 또는 시연 배치 기준으로 판단한다.
- COBOT target_zone_name은 하차/진열 위치 메시지와 작업 컨텍스트에 사용한다.
- 주문 상품 보관 위치는 PRODUCT_SLOT_*이다.
- PICKY가 해당 상품 앞에 정차하는 위치는 PRODUCT_ZONE_*이다.
- 진열 시 PICKY는 STOCK_ZONE에서 상품을 싣고 PRODUCT_ZONE_*으로 이동한다.
- 진열 시 COBOT은 창고 상품을 PICKY에 상차하고, 이후 진열 위치에 배치한다.
- pickup slot은 상품 상차가 모두 끝난 뒤 MOVE_TO_PICKUP 직전에 배정한다.

17. TrafficManager Interface

TrafficManager는 zone 기반 BFS 경로 탐색과 다중 PICKY 충돌 회피를 담당한다. 외부 I/O 없이 in-memory 상태만 관리한다.

PathResult:

```
1 @dataclass(frozen=True)
2 class PathResult:
3     ok: bool
4     waypoints: tuple[str, ...] = ()
5     cost: float | None = None
6     reason: str | None = None
```

주요 API:

메서드	목적
reserve_path(robot_id, task_id, source_zone, target_zone)	단일 목적지 이동 경로 예약
reserve_nearest_from(robot_id, task_id, source_zone, candidates)	후보 zone 중 최단 경로 선택 및 예약
attach_task_id(robot_id, task_id)	임시 예약에 task_id 연결
reserve_return_home_path(robot_id, task_id, source_zone)	standby zone 복귀 경로 예약
reserve_dock_path(robot_id, task_id, source_zone)	충전 도크 경로 및 도크 예약
update_path_progress(robot_id, task_id, current_waypoint_index)	통과한 경로 점유 해제
release_path(robot_id, task_id)	task 종료/실패/취소 시 경로 해제
notify_state(robot_id, state)	PICKY state 변경 반영 및 자동 해제 안전망

처리 원칙:

- TaskManager는 상품/픽업 후보를 결정한다.
- TrafficManager는 후보 중 가장 가까운 zone을 선택하고 경로를 예약한다.
- PathResult.waypoints[-1]이 선택된 목적 zone이다.
- 도크 점유 상태는 TrafficManager 내부 상태로 관리한다.

- action feedback의 current_waypoint_index로 지나온 구간을 해제한다.

18. 주문 task 흐름

주문 task는 상품별 이동/상차를 반복한 뒤 픽업존 이동/검수/하차로 마무리한다.

```

1  상품 1:
2    MOVE_TO_PRODUCT
3    SORTING_AND_LOAD
4
5  상품 2:
6    MOVE_TO_PRODUCT
7    SORTING_AND_LOAD
8
9  ...
10
11 모든 상품 상차 완료 후:
12   MOVE_TO_PICKUP
13   INSPECTION
14   UNLOAD

```

처리 원칙:

- Customer UI가 주문을 생성하면 order는 ORDER_WAIT 상태가 된다.
- 고객 음성 주문은 STT/LLM 처리 후 일반 주문 생성과 같은 흐름을 사용한다.
- 주문 생성 시점에는 task를 만들지 않는다.
- 주문 생성 시 상품 재고는 주문 수량만큼 차감된다.
- TaskManager가 ORDER_WAIT 주문을 polling으로 감지한다.
- TaskManager가 주문 item 목록을 관리한다.
- TaskManager가 남은 상품 후보를 {PRODUCT_ZONE_N: quantity} 형태로 TrafficManager에 전달한다.
- TrafficManager가 가장 가까운 상품 zone을 선택하고 path를 예약한다.
- TaskManager가 선택된 상품에 대해 MOVE_TO_PRODUCT / SORTING_AND_LOAD task를 생성한다.
- 모든 상품 상차가 끝나면 EMPTY pickup slot을 조회한다.
- TaskManager가 pickup zone 후보를 TrafficManager에 전달한다.
- TrafficManager가 가장 가까운 pickup zone을 선택한다.
- TaskManager가 같은 번호의 pickup slot을 주문에 배정한다.
- MOVE_TO_PICKUP / INSPECTION / UNLOAD task를 생성한다.
- UNLOAD SUCCESS 시 order=PICKUP_READY, pickup_slot=OCCUPIED로 전이한다.
- 고객 수령 완료 시 POST /api/orders/{order_id}/complete 로 order=COMPLETED, pickup slot은 EMPTY로 돌아간다.

19. 진열 task 흐름

진열 task는 자동 또는 관리자 UI 요청으로 생성된 display_item을 기준으로 TaskManager가 생성한다.

```
1 MOVE_TO_STOCK
2 SORTING_AND_LOAD
3 MOVE_TO_DISPLAY
4 DISPLAY_SCAN
5 DISPLAY_PLACE
```

처리 원칙:

- 진열 요청은 자동 진열 또는 관리자 UI 직접 요청으로 생성된다.
- 관리자 자연어 LLM 진열 명령은 사용하지 않는다.
- 자동 진열은 상품 재고가 부족 상태가 되었을 때 생성된다.
- 재고 기준은 기본 5개, 정상 4개 이상, 부족 임박 3개, 부족 2개 이하이다.
- 자동 진열 수량은 상품당 2개를 기본으로 한다.
- FleetRepository는 display_item을 REQUESTED 상태로 저장한다.
- 열린 진열 흐름이 있으면 새 display_item은 기존 display_batch_id를 재사용할 수 있다.
- TaskManager는 REQUESTED display_item을 polling으로 감지한다.
- TaskManager가 robot unit을 배정한다.
- TaskManager가 display_item 기준으로 진열 task 5개를 생성한다.
- task 생성 후 display_item은 ASSIGNED 상태가 된다.
- task 실행 중 display_item은 IN_PROGRESS 상태가 된다.
- DISPLAY_PLACE SUCCESS 후 product.stock_qty와 display_item 상태를 반영한다.
- processed_quantity는 COBOT이 실제 처리 완료한 수량이다.
- stock_delta는 실제 재고에 반영할 수량이다.
- 완료 시 display_item은 COMPLETED 상태가 된다.
- 여러 display_item이 같은 display_batch_id로 묶이면 관리자 UI에서는 하나의 진열 흐름처럼 표시될 수 있다.
- 실제 로봇 수행은 display_item 단위로 순차 처리한다.

20. 작업 종료 후 복귀/도킹/충전 흐름

주문 또는 진열 작업이 끝난 뒤 TaskManager가 다음 작업 여부와 배터리 상태를 보고 복귀/도킹/충전을 결정한다.

```
1 UNLOAD 또는 DISPLAY_PLACE 완료
2 -> 대기 주문/진열이 있으면 다음 작업 수행
3 -> 대기 작업이 없으면 RETURN_HOME
4 -> standby zone 도착
5 -> 필요하면 DOCK_IN
```

task_type	의미	TrafficManager 호출
RETURN_HOME	standby zone 복귀	reserve_return_home_path()
DOCK_IN	충전 도크 정밀 진입	reserve_dock_path()
CHARGE	충전 상태 진입/유지	경로 예약 없음

처리 원칙:

- RETURN_HOME은 standby zone 도착까지만 담당한다.
- DOCK_IN은 충전 dock 선택과 정밀 도킹을 담당한다.
- CHARGE는 충전 상태 진입/유지로 본다.
- 배터리가 기준치 이상으로 회복되면 TaskManager가 CHARGE SUCCESS를 처리한다.

21. ENUM 요약

order_status:

```
1 ORDER_RECEIVED
2 ORDER_WAIT
3 SORTING
4 DELIVERING
5 INSPECTING
6 PICKUP_READY
7 COMPLETED
8 ERROR
```

order_item_status:

```
1 WAITING
2 SORTED
3 INSPECTED
4 MISSING
5 EXCESS
6 MISMATCH
```

pickup_slot_status:

```
1 EMPTY
2 RESERVED
3 OCCUPIED
4 BLOCKED
```

robot_type:

```
1 PICKY
2 COBOT
```

robot_status:

1	OFFLINE
2	IDLE
3	BUSY
4	CHARGING
5	EMERGENCY_STOP
6	ERROR

picky_state:

1	CHARGING
2	STANDBY
3	MOVING_TO_PRODUCT
4	WAITING_FOR_COBOT
5	MOVING_TO_PICKUP
6	MOVING_TO_STOCK
7	MOVING_TO_DISPLAY
8	RETURNING
9	DOCKING
10	ERROR_RECOVERY

cobot_state:

1	STANDBY
2	SORTING
3	LOADING
4	INSPECTING
5	UNLOADING
6	SCANNING
7	PLACING
8	STOWING_ARM
9	SAFETY_STOPPED

task_type:

1	MOVE_TO_PRODUCT
2	SORTING_AND_LOAD
3	MOVE_TO_PICKUP
4	INSPECTION
5	UNLOAD
6	MOVE_TO_STOCK
7	MOVE_TO_DISPLAY
8	DISPLAY_SCAN
9	DISPLAY_PLACE
10	RETURN_HOME
11	DOCK_IN
12	CHARGE

task_status:

1	QUEUED
2	ASSIGNED
3	RUNNING
4	PAUSED
5	SUCCESS
6	FAILED
7	CANCELLED

display_policy:

```
1 REQUESTED_QUANTITY
2 ALL_PROCESSED
```

display_item_status:

```
1 REQUESTED
2 ASSIGNED
3 IN_PROGRESS
4 COMPLETED
5 FAILED
6 CANCELLED
```

exception_type:

```
1 OBSTACLE_DETECTED
2 LOW_BATTERY
3 NAVIGATION_FAILED
4 HARDWARE_ERROR
5 TIMEOUT
6 SORTING_FAIL
7 INSPECTION_FAIL
8 HUMAN_DETECTED
9 SYSTEM_ERROR
```

llm_action:

```
1 ORDER
2 CHAT
```

22. 금지/주의 항목

항목	이유
UI가 DB에 직접 접근	DB 소유권은 Fleet Manager/FleetRepository에 있다.
Web Gateway가 DB에 직접 접근	Web Gateway는 화면과 프록시만 담당한다.
PICKY/COBOT이 Fleet API task를 직접 조회	task 생성/배정/상태 전이는 Fleet Manager 책임이다.
Web Gateway가 직접 PICKY/COBOT에 이동/작업 명령 전송	로봇 제어 책임은 Fleet Manager와 RobotCommandGateway에 있다.
Web Gateway/Fleet API가 이미지/영상 중계	이미지/영상은 Vision 요청 주체가 AI Vision Server에 직접 보낸다.
주문 생성 시 바로 task 생성	task 생성은 TaskManager 책임이다.

주문/진열 감지를 push 필수로 의존	현재 기준은 TaskManager polling이다.
관리자 자연어 LLM 진열 명령	현재 LLM은 고객 음성 주문에서만 사용한다.
display_batch_id를 로봇 Action goal에 전달	display_batch_id는 Fleet DB/UI 표시용 흐름 그룹이다. 로봇은 display_item_id 기준으로 작업한다.
task_type=PATROL	순찰 시나리오 제거로 사용하지 않는다.